SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

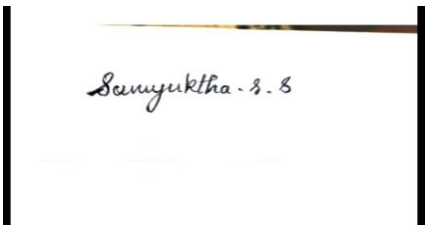
4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I Samyuktha S S(192311056) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

ANSWERS

Question 1: Reverse Engineering an ATM Withdrawal Module

1. Introduction

An ATM withdrawal module allows customers to withdraw money from their bank accounts. Since only the input-output behavior is available, reverse engineering is used to identify the module's hidden validation rules and reconstruct its testing strategy. Black-box testing techniques such as Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) are useful for this purpose.

2. Functional Decomposition

The ATM withdrawal module can be divided into the following components:

1. User Authentication: Verifies the PIN and identifies the customer.
2. Account Validation: Checks whether the account is active and accessible.
3. Withdrawal Amount Validation: Ensures the amount is within the permitted limits.
4. Balance Validation: Checks whether sufficient funds are available.
5. Cash Availability: Verifies whether the ATM has enough cash.
6. Transaction Processing: Deducts the amount and records the transaction.
7. Error Handling: Displays suitable messages for invalid PINs, insufficient balances, and other failures.

3. Equivalence Class Partitioning

ECP divides input data into valid and invalid groups. One representative value from each group is tested.

Input condition	Equivalence class	Expected result
Withdrawal amount	Valid amount	Transaction accepted if other conditions pass
Withdrawal amount	Zero or negative	Rejected
Withdrawal amount	Below minimum limit	Rejected
Withdrawal amount	Above maximum limit	Rejected
Account balance	Balance sufficient	Withdrawal may proceed
Account balance	Insufficient balance	Transaction rejected
PIN	Correct PIN	Authentication succeeds
PIN	Incorrect PIN	Authentication fails

The actual limits and validation rules must be confirmed from observed system behavior or requirements.

4. Boundary Value Analysis

BVA focuses on values at and around input limits. If the permitted withdrawal range is assumed to be ₹100 to ₹50,000, the following tests can be used.

Test value	Expected result
₹99	Reject
₹100	Accept if other conditions pass
₹101	Accept if other conditions pass
₹49,999	Accept if other conditions pass
₹50,000	Accept if other conditions pass
₹50,001	Reject

These limits are illustrative assumptions and should be replaced if the actual ATM uses different limits.

5. Improved Test Cases

- Enter a correct PIN and withdraw a valid amount.
- Enter an incorrect PIN and verify that access is denied.
- Withdraw zero or a negative amount.
- Test the minimum and maximum permitted withdrawal amounts.
- Attempt to withdraw more than the available balance.
- Attempt withdrawal when the ATM has insufficient cash.
- Interrupt the transaction and verify that the balance is not incorrectly deducted.
- Verify that successful and failed transactions are recorded correctly.

6. Evaluation of Test Adequacy

Testing only successful withdrawals is inadequate because it does not cover invalid inputs, boundary conditions, or exceptional situations. Combining ECP and BVA improves coverage by checking representative input groups and limit values. Additional tests for authentication, cash availability, and transaction failure improve reliability.

Conclusion

Reverse engineering the ATM module helps reconstruct its likely black-box testing strategy. Applying ECP, BVA, and functional testing provides systematic coverage of valid, invalid, and boundary scenarios, helping prevent incorrect withdrawals and balance calculation errors.

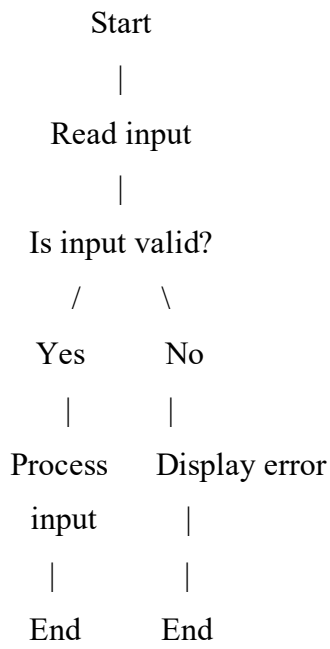
Question 2: Reverse Engineering White-Box Testing Using a Control Flow Graph

1. Introduction

White-box testing examines the internal logic, control structures, and execution paths of a software module. When test documentation is unavailable, execution traces and code snippets can be analyzed to reconstruct the original testing approach. A Control Flow Graph (CFG) helps identify decision points, loops, and independent execution paths.

2. Identifying Control Structures

Consider the following sample logic for a software module:



This is an illustrative example because the question does not provide a specific code snippet.

The important control structures are:

1. Sequential statements: Instructions executed one after another.
2. Decision points: Conditions that create alternative execution paths.
3. Loops: Statements repeated while a condition remains true.
4. Exception handling: Logic used to manage unexpected situations.

3. Control Flow Graph (CFG)

A CFG represents program execution using nodes and directed edges.

- Nodes: Represent statements or blocks of statements.
- Edges: Represent the possible flow of control.
- Decision nodes: Represent conditions such as if, else, and loop conditions.
- Entry and exit nodes: Represent the beginning and end of execution.

The CFG helps testers understand how different inputs affect program execution.

4. Independent Execution Paths

For the sample logic, two basic paths exist:

- Path 1: Start → Read input → Input valid → Process input → End.
- Path 2: Start → Read input → Input invalid → Display error → End.

If the actual program contains loops and additional conditions, more independent paths must be identified.

5. Coverage Criteria

Statement coverage: Ensures every executable statement runs at least once.

Branch coverage: Ensures every decision outcome, such as True and False, is executed.

Path coverage: Attempts to execute the independent paths through the program. Complete path coverage may be impractical when loops create infinitely many possible paths.

Condition coverage: Ensures each individual Boolean condition takes both True and False values.

Statement coverage alone may miss defects in decision logic. Therefore, branch coverage and condition testing should also be considered.

6. Reconstructed Testing Strategy

1. Identify all statements and decision points.
2. Construct the CFG from the available code.
3. Calculate cyclomatic complexity.
4. Identify independent execution paths.
5. Design test inputs to execute each branch.
6. Test loops for zero, one, and multiple iterations where applicable.
7. Compare actual execution traces with expected results.
8. Repeat tests after fixing defects to check for regressions.

Conclusion

A CFG provides a structured representation of a program's execution flow. By identifying independent paths and measuring statement, branch, and condition coverage, testers can reconstruct a more complete white-box testing strategy and reduce the risk of hidden logic errors.

Question 3: Reverse Engineering a Web Application Login and Transaction System

1. Introduction

A web application login and transaction system includes authentication, input validation, transaction processing, and result reporting. When only defect logs and test results are available, testers can analyze recurring failures to infer which testing techniques were previously used and identify missing coverage.

2. Functional Decomposition

The application can be divided into these components:

1. Login Module: Accepts usernames and passwords.
2. Authentication Module: Verifies credentials and manages access.
3. Session Management: Maintains user sessions and handles expiration.
4. Transaction Validation: Checks transaction amount, account status, and available balance.
5. Transaction Processing: Executes valid transactions and updates records.
6. Error Handling: Displays appropriate error messages.
7. Database Interaction: Stores user, account, and transaction information.

3. Identifying Existing Testing Techniques

Defect patterns may indicate the techniques used previously.

- Black-box testing: Likely used if defects were identified through incorrect outputs, invalid login responses, or transaction failures.
- Equivalence Class Partitioning (ECP): May have been used if testers selected representative valid and invalid inputs.
- Boundary Value Analysis (BVA): May have been used if defects were concentrated around minimum or maximum transaction amounts.
- White-box testing: May have been used if execution paths, internal conditions, or code branches were examined.

However, defect logs alone cannot prove which techniques were used. They provide evidence for reasonable inferences.

4. Equivalence Class Partitioning

ECP divides input values into groups that should behave similarly.

Input	**Valid class**	**Invalid class**
Username	Existing, valid username	Empty or unknown username
Password	Correct password	Incorrect or empty password
Transaction amount	Positive amount within limits	Zero, negative, or out-of-range amount
Account balance	Sufficient balance	Insufficient balance
Session	Active session	Expired or invalid session

A representative test should be selected from each class.

5. Boundary Value Analysis

Suppose the permitted transaction amount is assumed to be ₹100–₹50,000.

Test values should include ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001.

The expected behavior must follow the application's actual requirements. Tests should also examine the minimum balance, maximum transaction count, and session timeout boundaries where applicable.

6. Path Testing

Path testing examines the internal routes through the application. Important paths include:

- Successful login → Valid transaction → Successful database update.
- Failed login → Error message → Access denied.
- Successful login → Insufficient balance → Transaction rejected.
- Successful login → Invalid amount → Validation error.
- Successful login → Database failure → Safe error handling.
- Expired session → Transaction request → Reauthentication required.

These paths should be verified using code analysis and execution traces where available.

7. Comprehensive Testing Strategy

1. Apply ECP to login credentials and transaction inputs.
2. Apply BVA to transaction limits and session timeouts.
3. Use branch and path testing for authentication and transaction decisions.
4. Verify that failed transactions do not incorrectly modify account balances.
5. Test duplicate requests and interrupted transactions.
6. Check error messages and transaction logs.
7. Perform regression testing after fixing defects.

Conclusion

Combining black-box and white-box techniques provides stronger coverage than relying on either technique alone. ECP and BVA test input behavior, while path testing examines internal execution logic. The reconstructed strategy improves authentication reliability, transaction accuracy, and error handling.

Question 4: Reverse Engineering a Legacy Inventory Management System

1. Introduction

An inventory management system maintains stock quantities, processes stock updates, and generates low-stock alerts. When source code is unavailable, successful and failed transaction logs can reveal the system's hidden business rules, input boundaries, and validation conditions.

2. Functional Decomposition

The stock update process can be divided into:

1. Product Identification: Verifies that the product exists.

2. Stock Input Validation: Checks whether the quantity is valid.
3. Stock Update Processing: Adds or removes stock.
4. Stock Limit Validation: Prevents negative stock or invalid quantities.
5. Low-Stock Detection: Compares the quantity with a predefined threshold.
6. Alert Generation: Produces notifications when stock is low.
7. Transaction Logging: Records successful and failed updates.

3. Analyzing Transaction Logs

Successful and failed logs can be compared to identify possible rules.

For example:

- If negative quantities fail, the system probably rejects invalid stock values.
- If updates for unknown product IDs fail, product existence is validated.
- If alerts appear below a particular stock quantity, a low-stock threshold is likely implemented.
- If zero quantities are rejected, the system may require a positive stock update.

These are inferred rules; actual behavior should be confirmed against the available logs and requirements.

4. Equivalence Class Partitioning

Stock quantities can be divided into the following classes.

Input condition	Equivalence class	Expected result
Quantity	Positive valid quantity	Update accepted
Quantity	Zero	Accepted or rejected according to requirements
Quantity	Negative value	Rejected
Product ID	Existing product	Update may proceed
Product ID	Unknown product	Rejected
Stock level	Above alert threshold	No low-stock alert
Stock level	At or below alert threshold	Alert generated if the rule uses this condition

5. Boundary Value Analysis

Assume the low-stock threshold is 10 units. The following tests can help identify the alert boundary.

Stock quantity	Expected result
9	Low-stock alert
10	Alert if the rule is stock less than or equal to 10
11	No low-stock alert

To test stock validation, additional cases should include negative quantities, zero, and the maximum permitted quantity, if defined.

6. Reconstructed Black-Box Test Cases

1. Update stock for an existing product with a valid quantity.
2. Attempt an update for an unknown product.
3. Enter a negative quantity and verify rejection.
4. Test zero quantity according to the specified business rule.
5. Reduce stock below the low-stock threshold.
6. Set stock exactly at the threshold.
7. Set stock just above the threshold.
8. Verify that the displayed stock matches the recorded stock.
9. Confirm that failed updates do not alter the quantity.
10. Verify that each successful or failed operation is logged correctly.

7. Evaluation of Testing Gaps

Testing only successful stock updates may miss invalid inputs and alert-generation defects. Testing only quantities far above or below the threshold may also miss errors at the exact boundary. Combining ECP and BVA helps cover these gaps.

Conclusion

Reverse engineering transaction logs helps identify the likely business logic of a legacy inventory system. By dividing inputs into equivalence classes and testing boundary conditions, a systematic black-box test suite can be reconstructed to improve stock accuracy and low-stock alert reliability.

Question 5: Reverse Engineering a Library Management Module Using CPU and Memory Profiles

1. Introduction

A compiled executable file (.exe) and its CPU and memory profiles can provide clues about the internal behavior of a library management module. Execution spikes during stress testing may indicate repeated loops, complex decisions, large data processing operations, or memory-intensive activities.

However, performance profiles alone cannot prove the exact source-code structure. They must be combined with available logs, executable analysis, and controlled experiments.

2. Functional Decomposition

A library management module may contain the following components:

1. Book Search: Searches books using an ID, title, or author.

2. Book Issue: Validates a member and checks book availability.
3. Book Return: Updates the availability of returned books.
4. Fine Calculation: Calculates overdue charges.
5. Member Management: Validates membership details.
6. Database Access: Retrieves and updates book and member records.
7. Report Generation: Produces lists of books, members, and transactions.

3. Analyzing CPU and Memory Profiles

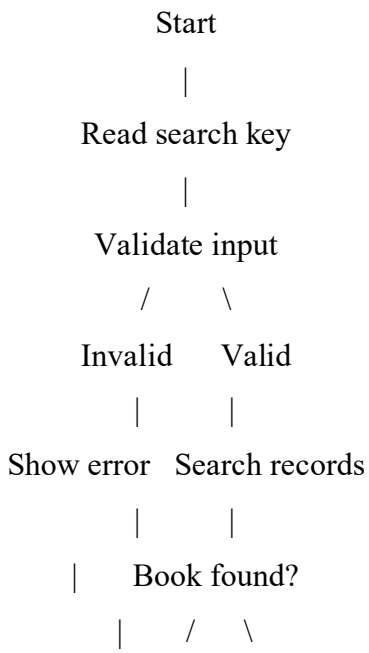
CPU and memory execution profiles can reveal possible performance problems.

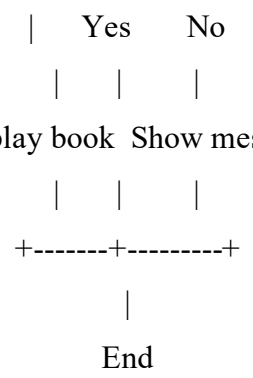
- High CPU usage during book searches: May indicate repeated searching or inefficient filtering.
- CPU spikes during fine calculations: May indicate repeated calculations or complex decision logic.
- Memory growth during report generation: May indicate large data collections being stored in memory.
- Increasing memory usage over time: May indicate memory leaks or objects that are not released.
- Long delays during database access: May indicate inefficient queries or excessive repeated access.

These patterns identify areas requiring further investigation rather than confirming a specific defect.

4. Conceptual Control Flow Graph

An illustrative CFG for book searching is shown below.





In an actual implementation, the search operation may include a loop that examines multiple records. Repeated iterations may contribute to CPU usage, especially with large datasets.

5. Reconstructing Path Testing

The following paths should be tested:

- Path 1: Valid search key → Book found → Display details.
- Path 2: Valid search key → Book not found → Display message.
- Path 3: Invalid or empty search key → Display validation error.
- Path 4: Valid issue request → Book available → Issue book.
- Path 5: Valid issue request → Book unavailable → Reject request.
- Path 6: Book return → Calculate overdue fine → Update records.
- Path 7: Large search or report request → Process many records → Return results.

The paths involving large datasets should receive special attention because they may contribute to high CPU or memory usage.

6. Stress Testing Strategy

1. Test book searches with small, medium, and large datasets.
2. Increase the number of simultaneous requests gradually.
3. Monitor CPU usage, memory consumption, response time, and errors.
4. Repeat searches for existing and nonexistent books.
5. Test issue and return operations under heavy load.
6. Generate large reports and observe memory growth.
7. Run repeated operations to check for memory leaks.
8. Verify that the system remains responsive and database records remain consistent.

7. Evaluating System Stability

The reconstructed tests help determine whether the module can handle increasing workloads without excessive delays, crashes, or incorrect results. Performance measurements should be compared

across repeated runs. If resource usage rises sharply, the relevant search, loop, or report-processing logic should be investigated and optimized.

Path testing verifies execution behavior, while stress testing measures performance under heavy workloads. Both are necessary because a program may achieve code coverage but still fail under resource-intensive conditions.

Conclusion

CPU and memory profiles can guide reverse engineering by identifying resource-intensive operations. A conceptual CFG and targeted path tests help investigate the suspected logic, while stress testing evaluates performance and stability. Together, these techniques improve the reliability of the library management module under heavy workloads.

Question 6: Reverse Engineering a Mobile Payment Gateway Using State Transition Testing

1. Introduction

A mobile payment gateway processes transactions through different states, such as Pending, Success, Reversal, and Timeout. Reverse engineering helps identify the state machine by examining API responses, transition triggers, and data constraints. State Transition Testing is used to verify valid and invalid transitions between these states.

2. Functional Decomposition

The payment gateway consists of the following components:

1. Request Validation: Checks transaction details and required fields.
2. Payment Initiation: Creates a transaction and sets its initial state.
3. Payment Processing: Communicates with the payment service.
4. Status Management: Updates the transaction state.
5. Reversal Handling: Reverses eligible transactions when required.
6. Timeout Handling: Handles transactions that do not receive a response in time.
7. Response Generation: Returns the transaction status through the API.

3. State Transition Analysis

The main states are:

- Pending: The transaction has started but is not yet completed.
- Success: The payment has been completed successfully.
- Reversal: A previously initiated or completed payment is being reversed according to the system's rules.
- Timeout: The transaction did not receive a response within the permitted time.

An illustrative state-transition model is:

Initiate

|

v

Pending

/ | \

Success Reversal Timeout

The exact legal transitions must be confirmed from the API responses and system requirements.

4. Reconstructed Test Cases

Test case	**Action**	**Expected result**
TC1	Initiate a valid payment	Pending
TC2	Receive payment confirmation	Success
TC3	Trigger a valid reversal	Reversal
TC4	Allow the request to time out	Timeout
TC5	Send a request with missing fields	Validation error
TC6	Attempt an illegal state transition	Request rejected or handled safely
TC7	Send a duplicate payment request	Duplicate payment prevented according to requirements
TC8	Retry after a timeout	Correct status returned without unintended duplicate payment

5. Testing Strategy

1. Identify all transaction states from API responses.
2. Identify the events that cause state changes.
3. Test every valid transition.
4. Attempt invalid transitions and verify safe rejection.
5. Test duplicate requests and retry behavior.
6. Verify transaction IDs, amounts, and status fields.
7. Check that the same transaction does not produce inconsistent states.
8. Test timeout and reversal scenarios under network failures.

Conclusion

State Transition Testing helps verify that a payment gateway moves between transaction states correctly. Reconstructing the state machine and testing both legal and illegal transitions improves transaction consistency, error handling, and payment reliability.

Question 7: Reverse Engineering an Encrypted Communication Module Using Fuzzing and BVA

1. Introduction

An encrypted communication module accepts input parameters and produces encrypted outputs. When its internal code is unavailable, testers can examine its behavior using fuzzing and Boundary Value Analysis (BVA). These techniques help identify invalid inputs, validation weaknesses, and possible security vulnerabilities.

2. Functional Decomposition

The module can be divided into:

1. Input Reception: Accepts user data and communication parameters.
2. Input Validation: Checks input length, format, and permitted characters.
3. Data Preparation: Converts accepted inputs into the required format.
4. Encryption: Encrypts the validated data.
5. Output Generation: Returns the encrypted result.
6. Error Handling: Rejects invalid inputs safely.

3. Fuzzing Analysis

Fuzzing involves supplying unexpected, malformed, or randomly generated inputs to observe how the module behaves.

Examples include:

- Empty input fields.
- Extremely long strings.
- Invalid characters and malformed data.
- Missing parameters.
- Incorrect data types.
- Repeated or unexpected special characters.
- Inputs containing unexpected control characters.

The tester should observe whether the module returns a controlled error, crashes, hangs, or produces incorrect output.

4. Boundary Value Analysis

Assume the module accepts input lengths from 1 to 1,024 bytes. These are illustrative limits and must be replaced with the actual specification.

Input length	Expected result
0 bytes	Reject if empty input is prohibited
1 byte	Accept if otherwise valid
2 bytes	Accept if otherwise valid
1,023 bytes	Accept if otherwise valid
1,024 bytes	Accept if otherwise valid
1,025 bytes	Reject if the maximum is 1,024 bytes

These tests examine whether the module handles the limits correctly.

5. Security-Oriented Test Cases

1. Submit valid input and verify successful encryption.
2. Submit empty or missing fields.
3. Submit input exceeding the permitted length.
4. Submit malformed data and invalid parameter types.
5. Send repeated requests to identify stability issues.
6. Verify that invalid inputs produce controlled errors.
7. Check that sensitive information is not exposed in error messages or logs.
8. Verify that oversized inputs do not cause crashes or memory corruption.

6. Evaluation of Testing Adequacy

Testing only normal inputs is insufficient because malformed and oversized inputs may expose weaknesses. Combining fuzzing with BVA provides broader input coverage. Security testing should also examine input validation, safe memory handling, and resistance to injection attacks where applicable.

Conclusion

Fuzzing and BVA help reverse engineer the input validation behavior of an encrypted communication module. By testing normal, malformed, and boundary inputs, testers can identify weaknesses and improve reliability, robustness, and security.

Question 8: Reverse Engineering an Insurance Premium Calculator with White-Box Testing

1. Introduction

An insurance premium calculator determines premiums using conditions such as age, coverage amount, policy type, and risk category. A test suite with 100% statement coverage may still miss

defects in compound conditions. Therefore, branch coverage, condition coverage, and path testing are needed to improve test adequacy.

2. Functional Decomposition

The calculator can be divided into:

1. Input Validation: Checks age, coverage amount, and policy details.
2. Risk Classification: Assigns the customer to a risk category.
3. Premium Calculation: Computes the base premium.
4. Discount Application: Applies eligible discounts.
5. Surcharge Calculation: Adds charges when applicable.
6. Result Generation: Displays the final premium.

3. Example of a Compound Condition

Consider this illustrative logic:

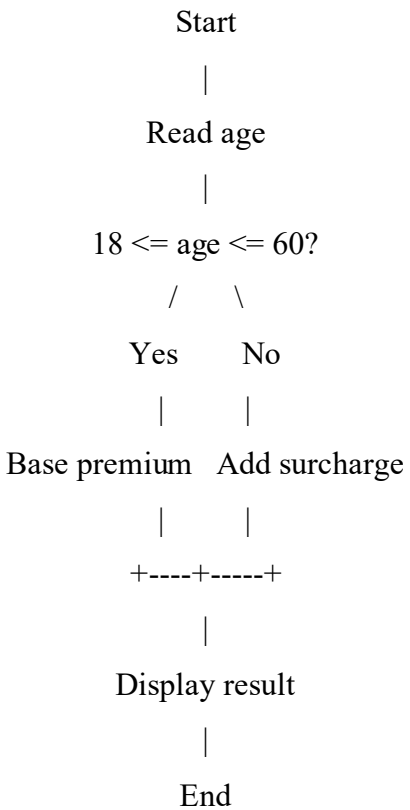
if age >= 18 and age <= 60: premium = base_premiumelse: premium = base_premium * 1.2

The condition contains two Boolean expressions:

- \(A\): age is greater than or equal to 18.
- \(B\): age is less than or equal to 60.

The overall condition is \(A \land B\).

4. Control Flow Graph



5. Test Cases

Test case	Age	Expected result
TC1	30	Base premium
TC2	17	Premium with surcharge
TC3	61	Premium with surcharge
TC4	18	Base premium
TC5	60	Base premium

These cases test the valid range and its boundaries.

6. Coverage Evaluation

Statement coverage: Checks whether every executable statement runs. It does not guarantee that every decision outcome is tested.

Branch coverage: Requires both True and False outcomes of the decision to execute.

Condition coverage: Requires each individual Boolean condition to evaluate to True and False.

Path testing: Tests independent execution paths through the program.

For the example, the valid-age and invalid-age branches must both be executed. More complex calculators require additional tests for discounts, surcharges, and combinations of conditions.

Cyclomatic complexity can be estimated using:

$$ V(G)=E-N+2P $$

For a single connected control flow graph, $P=1$.

7. Improved Testing Strategy

1. Construct a CFG from the calculator's logic.
2. Identify all decision points and compound conditions.
3. Test values below, at, and above each boundary.
4. Ensure every branch is executed.
5. Test combinations of age, policy type, and coverage amount.
6. Verify premium calculations against expected results.
7. Repeat tests after correcting defects.

Conclusion

Statement coverage alone cannot guarantee that all decision logic is correct. Combining branch coverage, condition coverage, BVA, and path testing improves the detection of errors in premium calculations and increases the reliability of the insurance application.

Question 9: Reverse Engineering a Cloud-Based File Synchronization Tool

1. Introduction

A cloud-based file synchronization tool transfers files between devices and cloud storage. Defects such as lost files may result from file size limitations, unsupported file types, network interruptions, or synchronization conflicts. Reverse engineering user activity logs and defect reports helps identify these weaknesses and reconstruct an improved testing strategy.

2. Functional Decomposition

The synchronization system contains the following components:

1. File Selection: Identifies files to synchronize.
2. Metadata Processing: Reads file names, types, and sizes.
3. Upload and Download: Transfers file contents.
4. Network Management: Handles slow connections and interruptions.
5. Conflict Detection: Identifies files modified in multiple locations.
6. Synchronization Verification: Confirms that the transferred file is complete.
7. Error Recovery: Retries failed transfers and records errors.

3. Equivalence Class Partitioning

The inputs can be grouped into equivalence classes.

Input	Equivalence class	Expected behavior
File size	Small valid file	Synchronization succeeds
File size	File near maximum limit	Correct handling at the limit
File size	File exceeding the limit	Rejected or handled according to requirements
File type	Supported type	Synchronization succeeds
File type	Unsupported type	Clear error or rejection
Network speed	Stable connection	Normal transfer
Network speed	Slow connection	Transfer completes or times out safely
Network status	Interrupted connection	Recovery without data loss

4. Boundary Value Analysis

If the system has a maximum file size of 100 MB, test:

- 99 MB
- 99.9 MB
- 100 MB
- 100.1 MB
- 101 MB

The expected result depends on the documented size limit and measurement units.

Network tests should also examine transfer timeouts and retry limits.

5. Reconstructed Test Cases

1. Synchronize a small file using a stable network.
2. Synchronize files near the maximum permitted size.
3. Attempt to synchronize an oversized file.
4. Test supported and unsupported file types.
5. Interrupt a transfer and restore the connection.
6. Modify the same file on two devices and check conflict handling.
7. Verify file contents and checksums after synchronization.
8. Repeat failed transfers and check for duplicate or incomplete files.
9. Run multiple transfers simultaneously.
10. Verify that error logs accurately record failed operations.

6. Stress Testing

Stress testing evaluates the system under heavy workloads, such as many simultaneous uploads, large files, and unstable network conditions. It helps identify performance limits, resource exhaustion, and synchronization failures.

The system should preserve file integrity even when a transfer fails. Temporary files should not replace valid files until the transfer is completed and verified.

7. Evaluation of Testing Adequacy

Testing only small files on stable networks is insufficient. Combining ECP, BVA, and stress testing covers different file categories, size limits, and workload conditions. Checksums and recovery tests provide additional evidence that files are not corrupted or lost.

Conclusion

Reverse engineering synchronization logs and defect reports helps identify likely causes of file loss. A revised strategy combining ECP, BVA, stress testing, and file-integrity verification improves synchronization reliability and protects data during network failures.

Question 10: Reverse Engineering an E-Commerce Search and Filter Module

1. Introduction

An e-commerce search and filter module allows users to find products based on criteria such as category, price, brand, and availability. By examining SQL queries and their resulting datasets, testers can infer the filtering logic and reconstruct a decision-table-based testing strategy.

2. Functional Decomposition

The search module can be divided into:

1. Search Input Processing: Reads keywords and filter parameters.

2. Parameter Validation: Checks input format and acceptable ranges.
3. Query Construction: Combines search criteria.
4. Database Execution: Runs the generated SQL query.
5. Result Filtering: Returns products matching the criteria.
6. Sorting and Pagination: Organizes results and displays them in pages.
7. Error Handling: Manages invalid filters and database errors.

3. Identifying Decision Logic

The SQL query may use the following logical operators:

- AND: A product must satisfy all combined conditions.
- OR: A product may satisfy at least one of the specified conditions.
- NOT: Excludes products satisfying a particular condition.

For example:

```
SELECT *
FROM Products
WHERE category = 'Electronics'
  AND price <= 50000
  AND available = 1;
```

This query retrieves available electronic products priced at or below ₹50,000.

By comparing query parameters with the returned products, testers can infer whether the conditions are applied correctly.

4. Decision Table

Consider three conditions:

- C1: Product belongs to the selected category.
- C2: Product is within the price limit.
- C3: Product is available.

Rule	C1	C2	C3	Expected result
R1	True	True	True	Include product
R2	False	True	True	Exclude product
R3	True	False	True	Exclude product
R4	True	True	False	Exclude product
R5	False	False	True	Exclude product
R6	False	True	False	Exclude product
R7	True	False	False	Exclude product

R8	False	False	False	Exclude product

This table represents the behavior of a query in which all three conditions are combined using AND.

5. Reconstructed Test Cases

1. Search using a valid product keyword.
2. Filter by category only.
3. Filter by maximum price only.
4. Filter by availability only.
5. Apply all three filters together.
6. Test products exactly at the maximum price boundary.
7. Enter an invalid or negative price.
8. Search for a keyword with no matching products.
9. Combine filters using OR or exclusion rules if supported.
10. Verify sorting, pagination, and empty-result handling.

6. Evaluation of Completeness

A decision table helps ensure that combinations of conditions are not overlooked. However, testing only the eight Boolean combinations may not cover every possible price boundary, invalid parameter, SQL error, or pagination issue.

Therefore, decision-table testing should be combined with ECP, BVA, input validation, and appropriate security testing. Query results must be checked against the expected dataset.

Conclusion

Reverse engineering SQL queries and their outputs helps reconstruct the search module's filtering logic. Decision tables systematically test combinations of conditions, while boundary and input-validation tests identify additional defects. This improves search accuracy, completeness, and reliability.